# TEAM LINEAR

EEE 4308 Project

Phase C, D and E (SW)

*Supervised by Mr. A. K. M. Rakib*

220021252-Yasin Hasan Saad
220021250- Tashfeea Ainur Rahman
220021248- Tanvir Hossain
220021242- Mubtasim Labib
220021208- Arham Takib
220021202- K.M. Tausif Ahnaf

## INTRODUCTION:

This report chronicles the development of Phase C, D and E of the software phase of the project for EEE4308. This also describes the incremental changes made to the schematic created during Phase A and B.

A two-player Snake-Ladder Game is to be made using electronic circuits.

The board will be a 4x4 grid, counting from 0 to 15. Each square will have 4 LED indicators.

At the start of the game, both players will be located at the square 0, which will be indicated by their respective LED indicators. A random number generator (the dice) will generate the numbers 1,2,3 randomly.

Players will take turn rolling the dice. The player rolling the will advance by n position, where n is the output of the dice.

There will be special positions on the board called Ladders and Snakes, each having two ends. Upon landing on the start point of the ladder, the player will be promoted to the other end of the ladder. In the case of the

snake, the player will be demoted. The starting positions and lengths of these will be specified by the user at the start of the game.

The player reaching the position 15 on the board first will win.

## REQUIREMENTS:

For Phase C, D and E, we were required to build the following:

### PHASE C:

- Dice Module

- High Frequency Clock Module (using 555 timer IC)

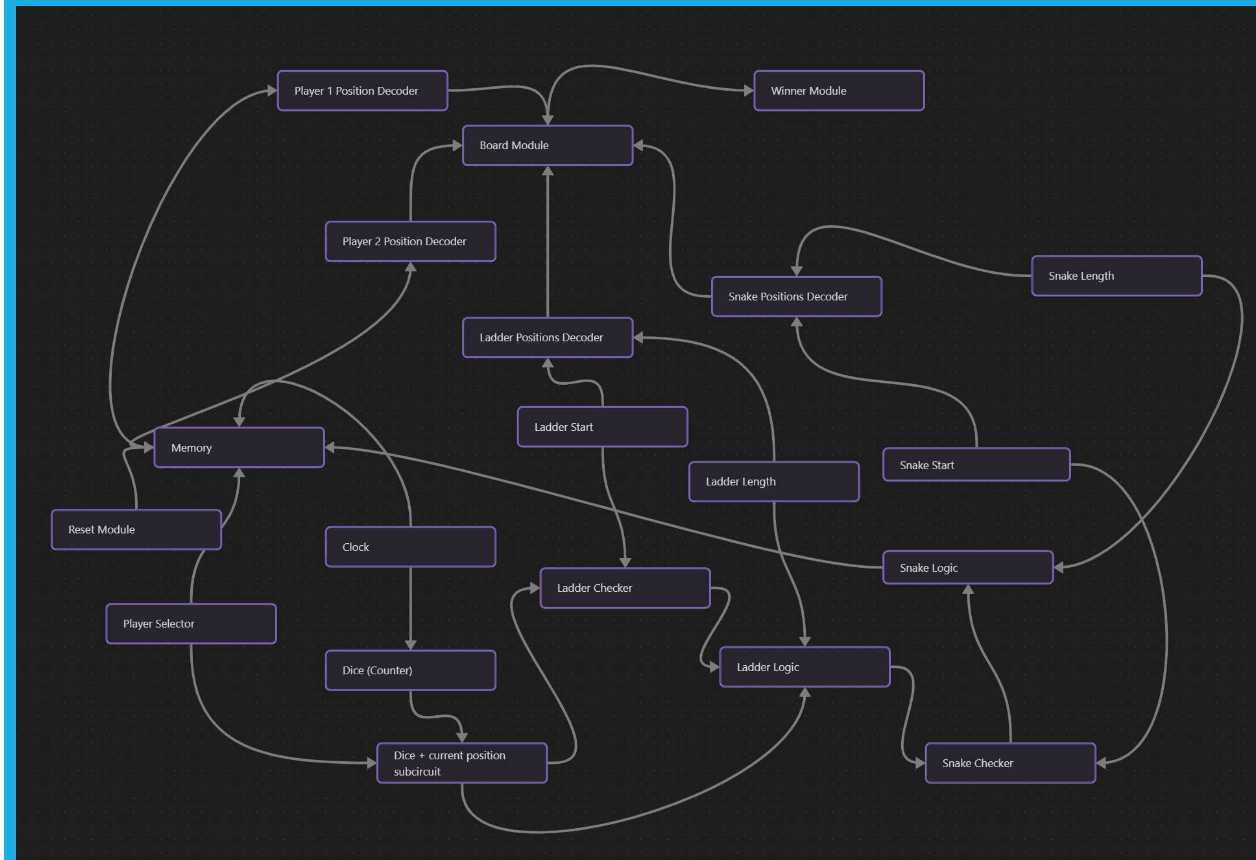
## PHASE D:

- Memory Module
- Reset Module
- Player Selector Module
- Winner Indicator Module

## PHASE E:

- Complete Integration of Digital Snake Ladder Game

# OVERVIEW



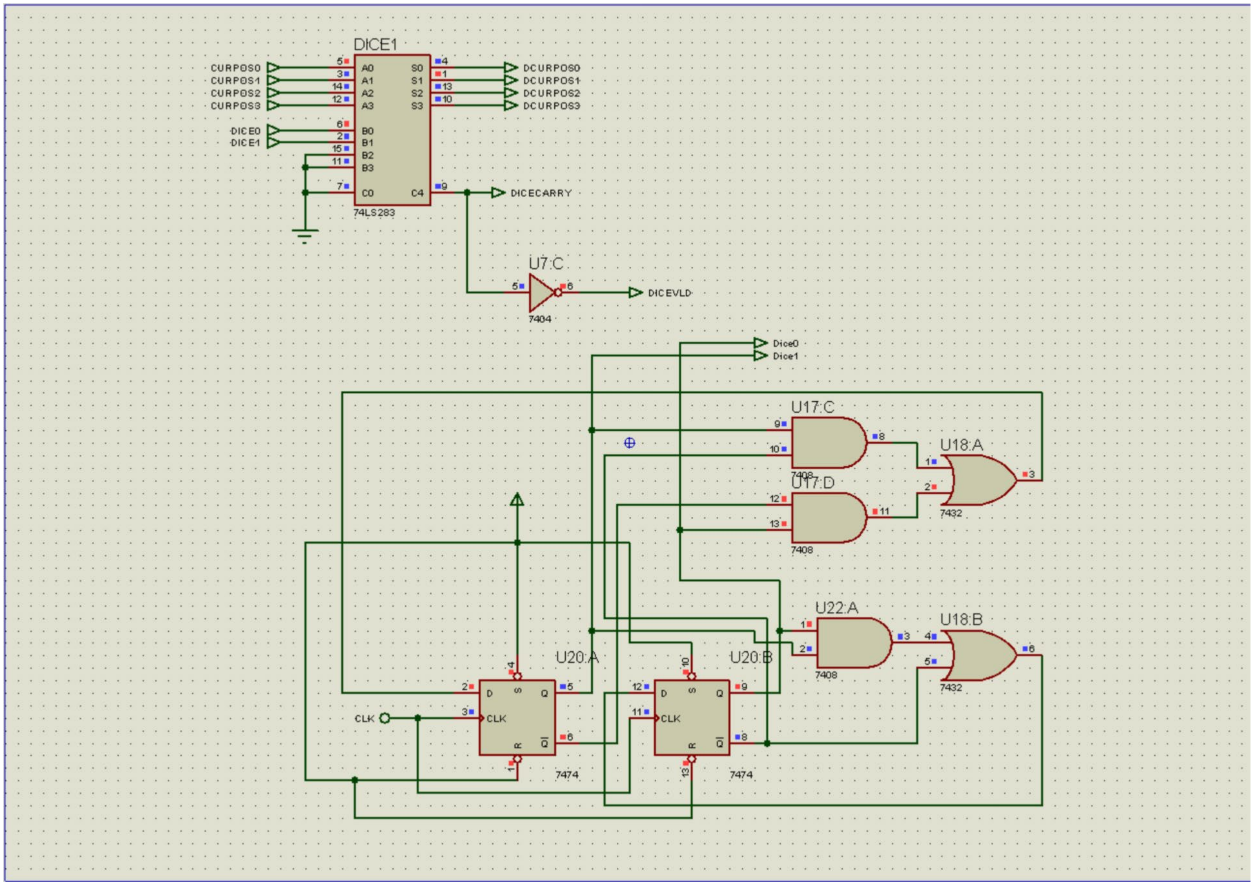

**i Interconnectivity of the Modules**

# PHASE C:

## DICE MODULE:

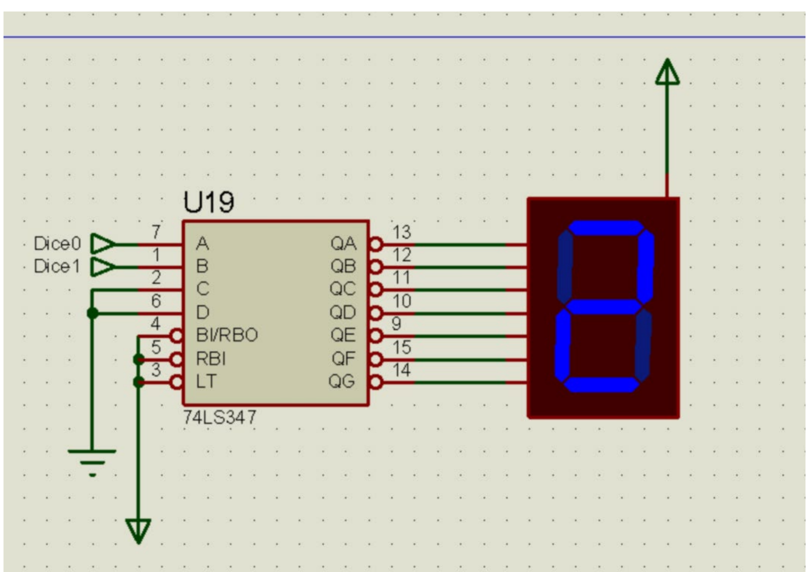
*The Dice Module can be found under the sheet titled "DICE".*

The dice is basically an up counter which skips the number 0. This is done by using a synchronous counter where state 00 has been skipped using combinatorial logic. A pair of JK Flipflops in toggle mode are used with a clock pulse from the clock module.




ii Dice Module

The states 01, 10, 11 are interpreted as 1, 2, 3 in the game.




iii Dice value shown with 7-seg-display

We used a 7-seg-display and a BCD-to-7-seg decoder to show the dice output. When the dice is “rolling”, the display also rolls between 1 and 3.

There is also an adder circuit that adds the dice output and the current position of the player (obtained from the memory module). This part was done in the previous phases and hence we won’t describe it in great detail here.

The DICEVLD signal checks whether the rolled output is a valid one (doesn’t exceed position 15).

**State Transition Table**

| Present State ($Q_0$ $Q_1$) | Next State ($Q'_0$ $Q'_1$) | $D_0$ | $D_1$ |
|---|---|---|---|
| **0 0** | 0 1 | 0 | 1 |
| **0 1** | 1 0 | 1 | 0 |
| **1 0** | 1 1 | 1 | 1 |
| **1 1** | 0 1 | 0 | 1 |

**Derived Expressions**

- **For D0**:

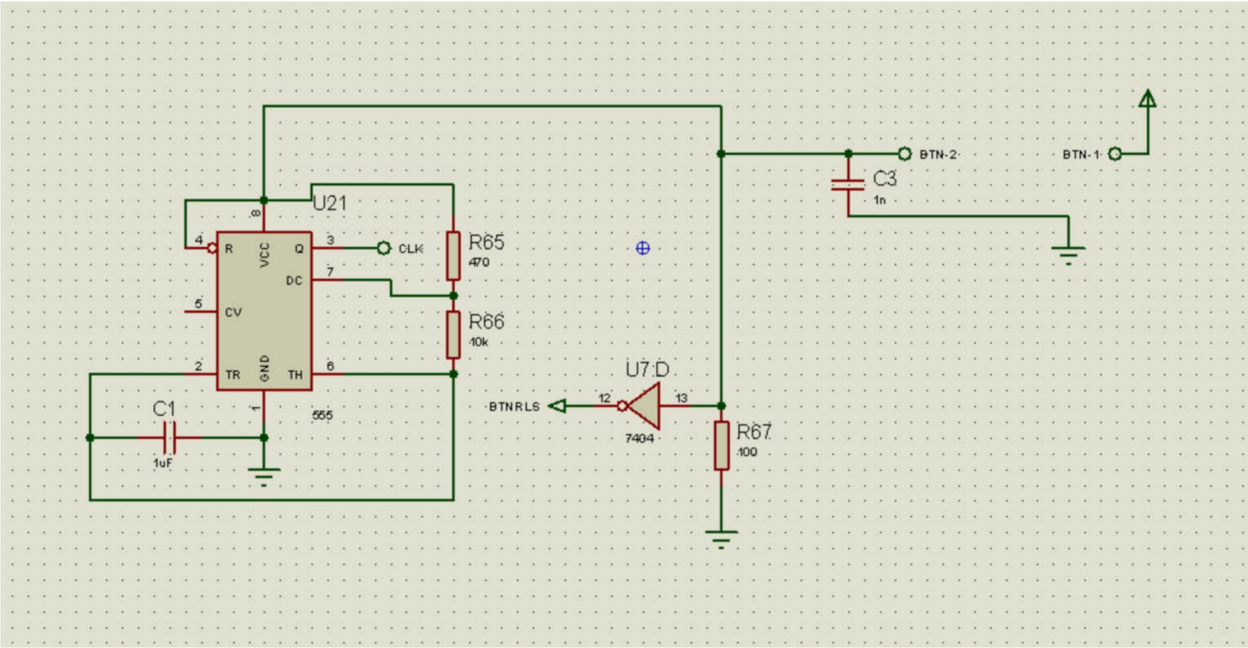
D0 = Q’0 Q1 + Q’1 Q0

- **For D1**:

D0 = Q1 + Q’1 Q’0

## HIGH FREQUENCY MEMORY CLOCK:

This module utilizes the astable mode of the 555 timer to obtain a clock pulse that is used to drive the memory and dice modules. The schematic can be found under the CLOCK sheet.




iv Clock Module

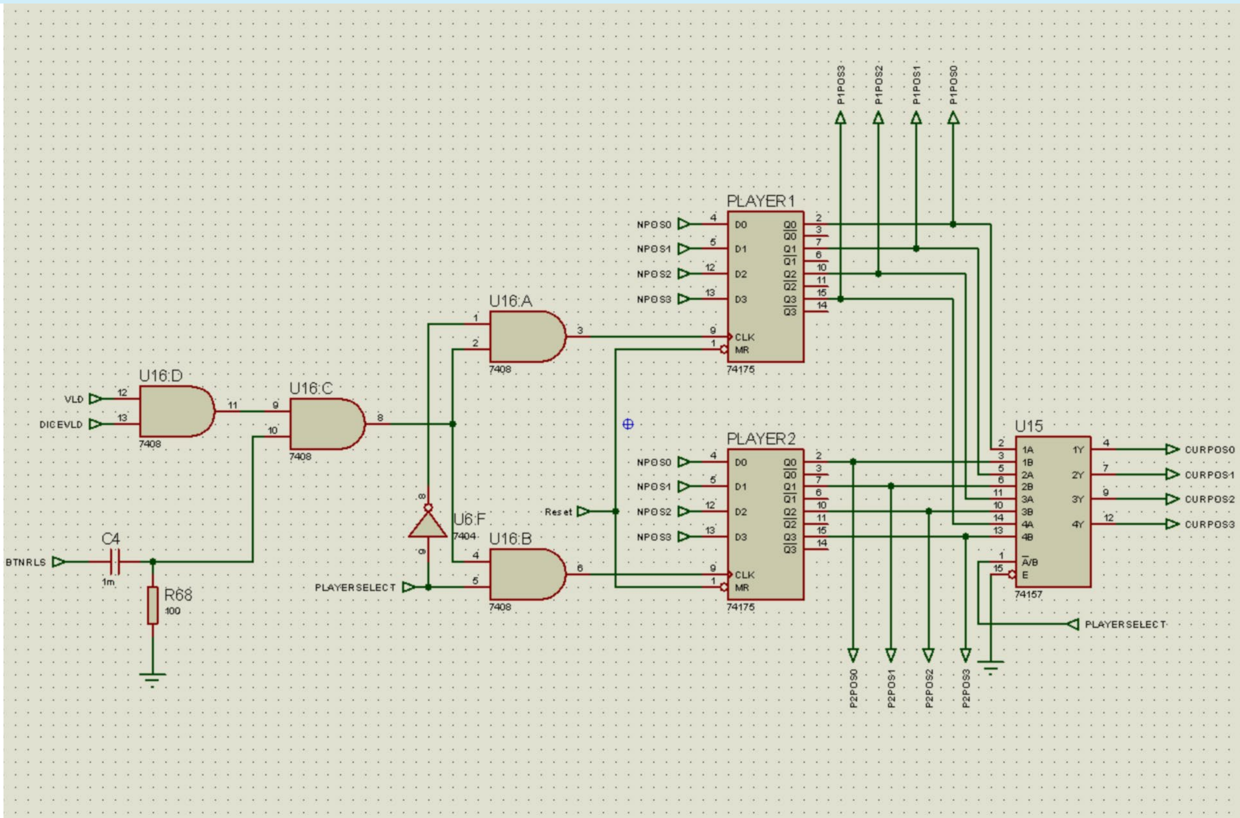
The frequency of the clock is given by $f = \frac{1.44}{(R_1+2R_2)C}$ = 137.5 Hz

The terminals BTN-1 and BTN-2 are normally open, they are closed once the dice button is pressed. This connects the clock module to power through VCC

There is also a terminal called BTNRLS here which detects when the button is released.

# PHASE D:

## MEMORY MODULE:




v Memory Module

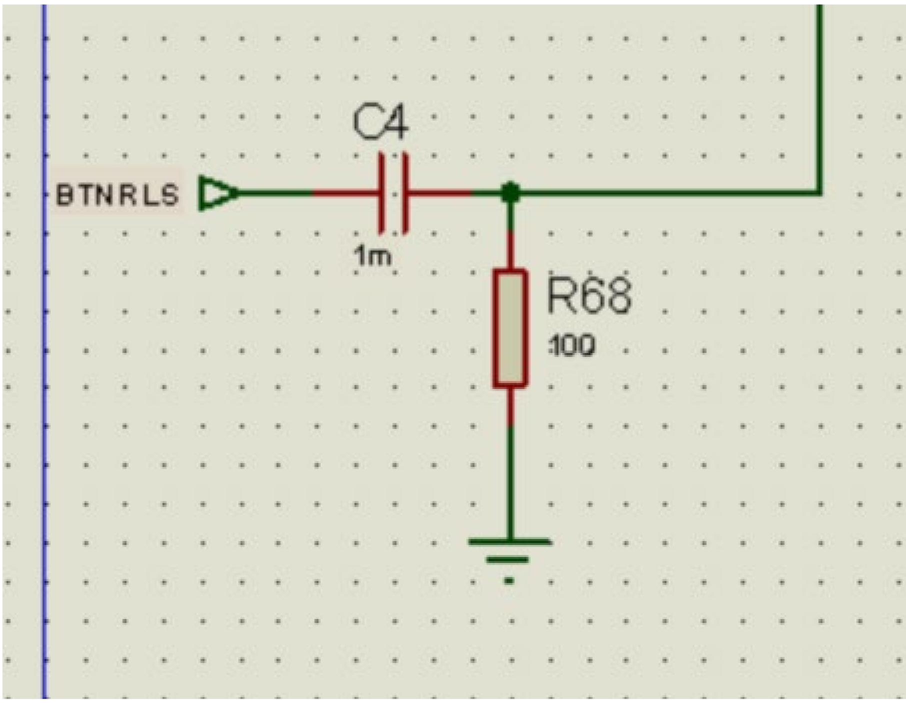
Memory Module can be found under the sheet “Memory”.

The main component is the IC 74175, which is a Quad D Type Flipflop. We used two of these, each storing positional data of each player.

For the Clock Pulse of this circuit, we utilized a variety of techniques to update the memory only when the dice button is released.

We first checked the Validity of the state of the game (Ladder, snake position and lengths are valid and the Dice roll is valid, meaning the dice roll doesn’t exceed the position 15) using an AND gate, and then the output signal is AND-ed with a edge trigger of the BTNRLS signal.

To achieve this edge trigger mechanism, we employed a simple RC network, that gives a sharp output curve.



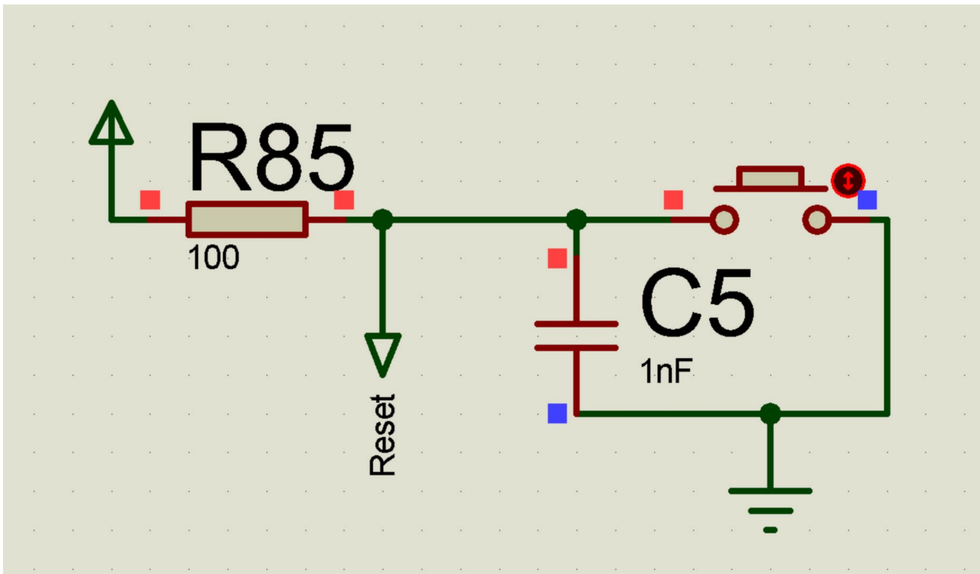

vi Edge detection network

The resultant pseudo-pulse is then fed into the correct memory IC using AND gates and output from the Player Selector.

The flipflops then update the memory from the dice adder output.

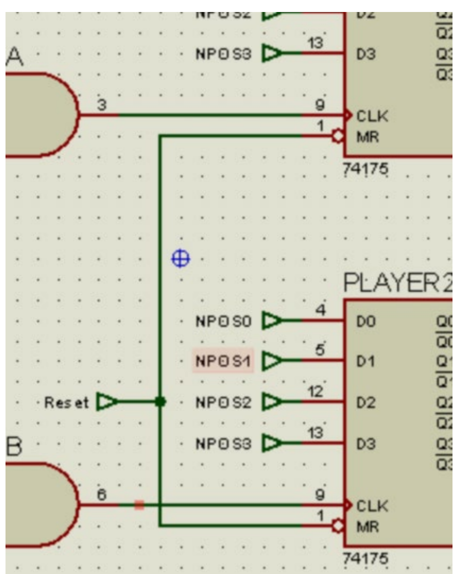
## RESET MODULE:

This is not a standalone module per-se, but rather a toggle that sends a signal to the Manual Reset pin of the flipflops, essentially setting them back to 0000.




vii Reset Button, Found in Sheet: Board Module

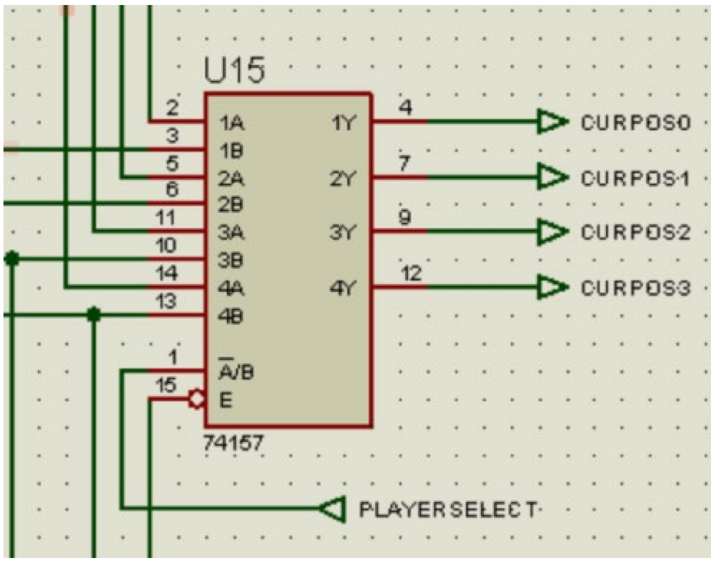
Since the MR pin is Active Low, we added a pull up resistor to the signal. Capacitor is added in order to debounce the button.



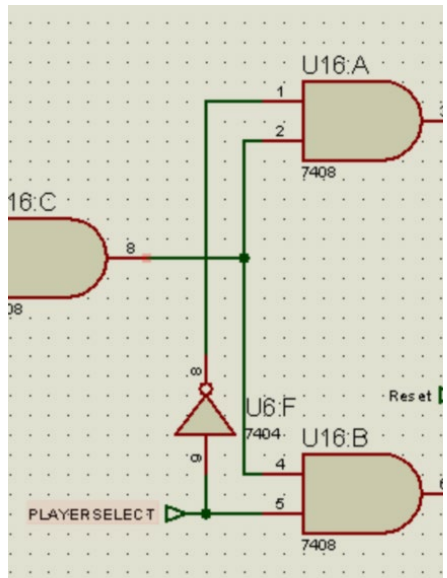

viii Reset Signal Fed into MR pin of 74175

## PLAYER SELECTOR MODULE:

This is also not a standalone module. It sends a signal to the memory input side, which, with the help of combinatorial logic circuit, determines which flipflop to update. This signal also controls the MUX (74157 in sheet Memory), which selects which data to send to the dice adder.



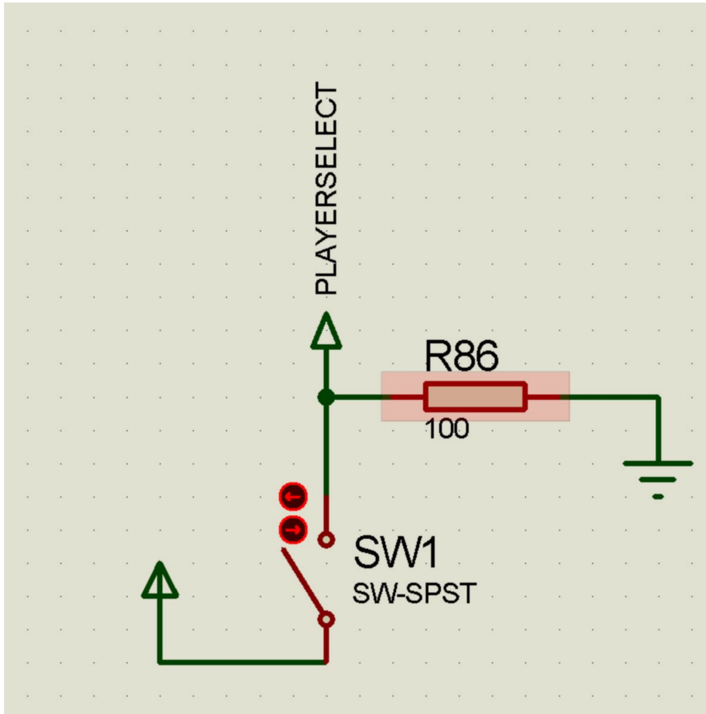

ix Multiplexer controlled by PlayerSelect signal




x PlayerSelect enabling appropriate memory cell

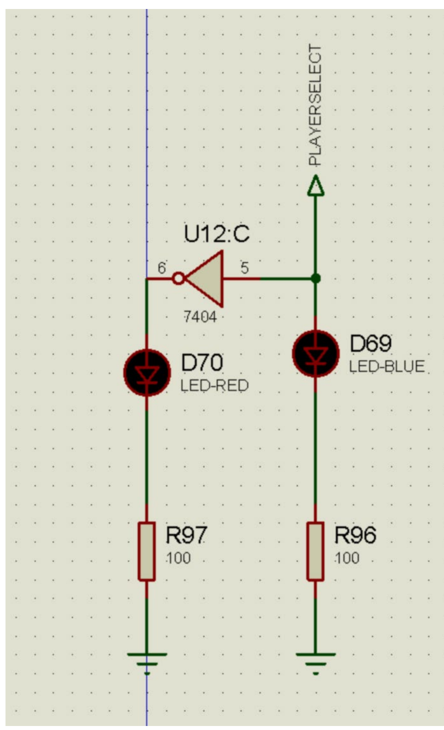
The signal itself comes manually from the user. The interface can be found under the sheet Board Module.

We wanted to make it an automatic toggle, but as the project manual suggested a physical switch, we decided to opt for that.




xi Player Selector interface with pull down resistor

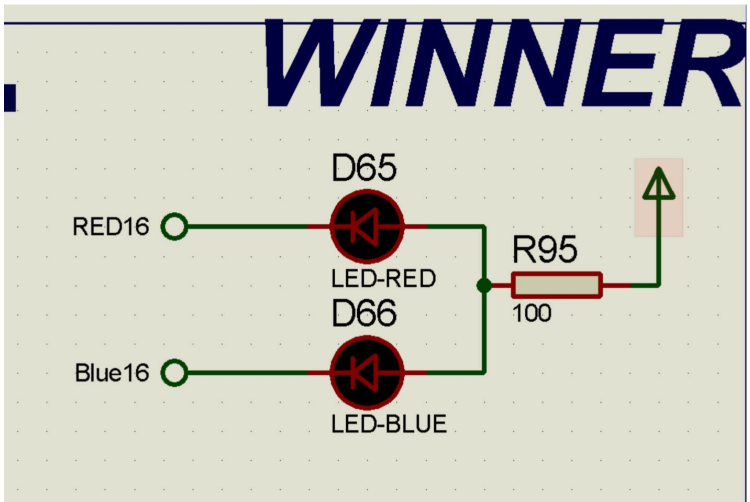
We have also incorporated an indicator to show which player is selected.




xii Player Selector Indicator

## WINNER INDICATOR MODULE:

This is also a simple addition to the game. Avoiding any complexity, we simply routed the LED signals from position 15 to another set of LEDs, which will light up once a player lands on that position.



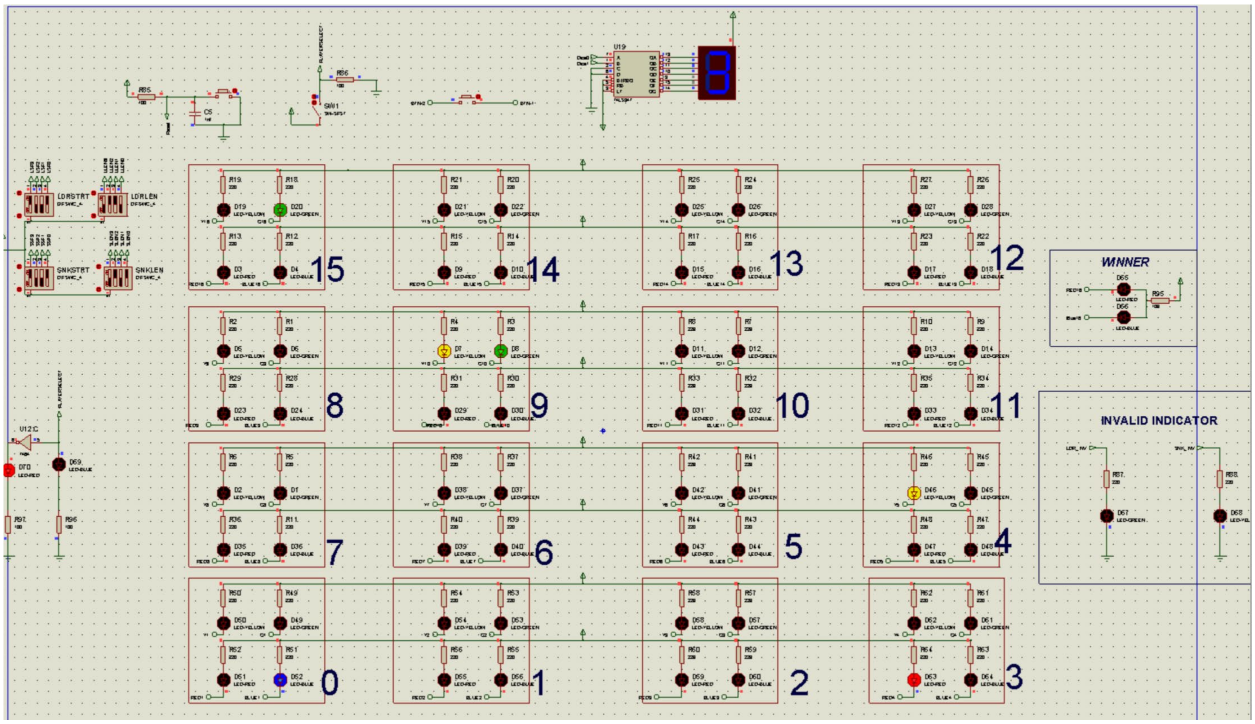

xiii This is also in the sheet Board Module

We wanted to add an auditory cue as well using a buzzer, but that proved to be very annoying and hence we decided to remove it.

# PHASE E:

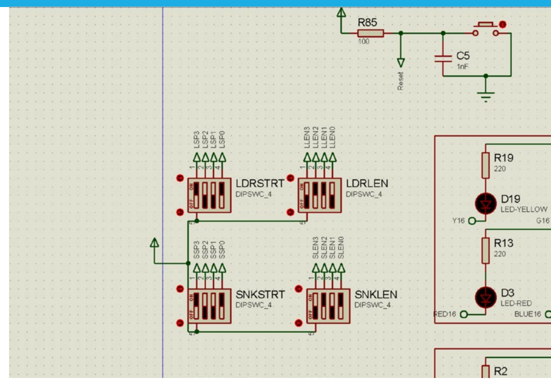
## COMPLETE INTEGRATION OF DIGITAL SNAKE LADDER GAME

Once all the modules have been finished and connected appropriately with the other modules, the game is operational.



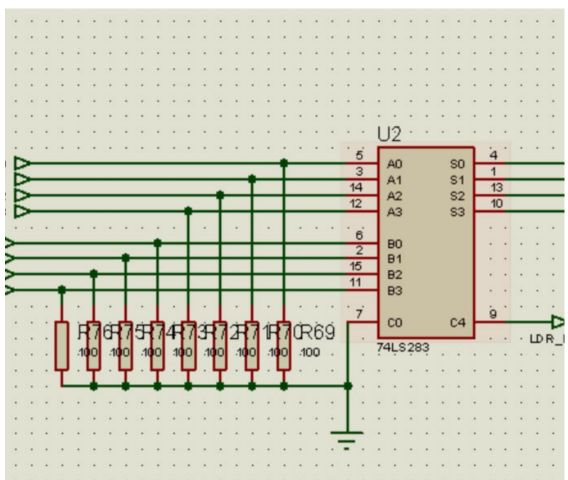

xiv Operational Game

## CHANGES MADE TO PREVIOUS SCHEMATICS

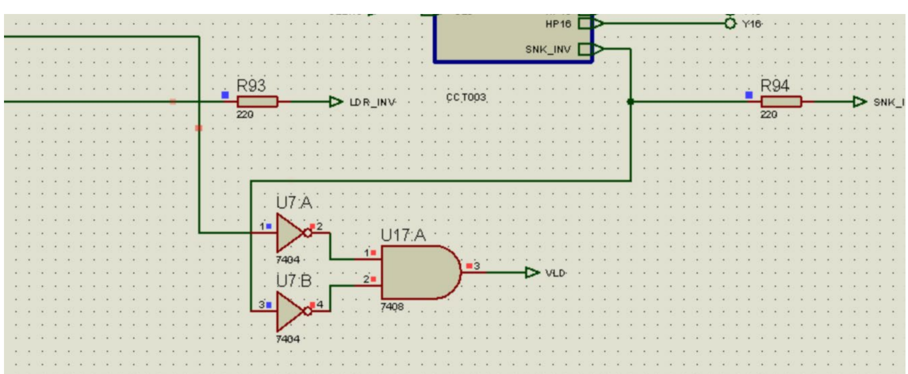



xv DIP switches to set Ladder and Snake parameters

To set ladder and snake parameters, we added DIP-4 switches. To facilitate this, we also added pull down resistors in the ladder and snake decoders.

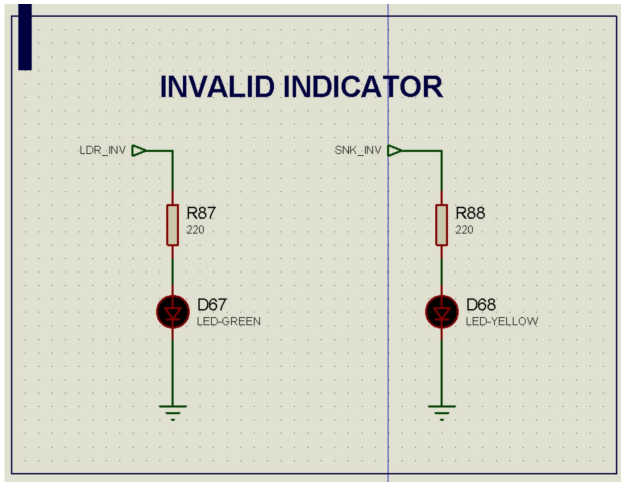



xvi Pull Down Resistors




xvii Combining two signals to make a “valid” signal

We also combined the ladder and snake invalidity signals to create a validity signals. We could have used a NOR gate instead, but we opted for this topology to minimize they types of ICs we are using.




xviii Invalid Indicator

We also added an indicator to indicate invalid ladder and snake parameters.

## ISSUES FACED

We faced a number of minor issues while doing these phases of the project. The most noteworthy one is a asynchronous counter not working in the main file but simulating properly in a separate file. We suspect that the issue was due to the complications becoming too “divergent” for proteus. The warning messages raised suggest the same thing.

Sometimes the simulation would become too slow and the simulation speed would be extremely low compared to the real time. This caused the dice to behave in an unwanted way. Restarting the software fixed this issue for us.

## COMPONENTS USED

1N4148
7SEG-COM-AN-BLUE
74LS283
74LS347
555
7404
7408
7432
7474
7485
74154
74157
74175
BUTTON
CAP
DIPSWC_4
LED-BLUE
LED-GREEN
LED-RED
LED-YELLOW
LOGICPROBE (BIG)
RES
SW-SPST

We used the components listed above.

# MEMBER CONTRIBUTIONS

## 220021202- K.M. TAUSIF AHNAF

- Intensively tested the individual modules to find errors and fix them
- Provided valuable insights to design the clock module with the 555 timer IC
- Kept track of the overall progress and took appropriate steps in response whenever the progress stagnated

## 220021208- ARHAM TAKIB

- Designed the synchronous counter for the dice module, with necessary K-maps and excitation tables
- Incorporated the timer IC to provide clock pulse to roll the dice
- Replaced the old asynchronous counter with the synchronous counter, which greatly reduced CPU load, and in some instances, convergence error during simulation
- Integrated the winner module and invalid input indicator
- Jointly integrated the whole project

## 220021242- MUBTASIM LABIB

- Suggested design choices keeping the PCB implementation in mind
- Checked the integrated project in order to find errors and fix them
- Kept track of the overall progress and took appropriate steps in response whenever the progress stagnated
- Asked questions to relevant faculties when needed

## 220021248- TANVIR HOSSAIN

- Provided the prototype design for the dice randomizer, which greatly enhanced our insights
- Designed the clock circuit using 555 timer to provide necessary clock pulses for the project
- Added a buzzer in a module precursor to the current winner module

## 220021250- TASHFEEA ALINUR RAHMAN

- Designed the winner module
- Actively checked up on the project for updates, ensuring work is done in a timely manner
- Extensively tested the integrated project to look for errors and fix them

## 220021252- YASIN HASAN SAAD

- Designed the Memory module and other related modules
- Worked on many prototypes before the team settling on the current modules in use
- Designed an edge detector to detect the release of the button
- Changed and added some of the previous modules to better facilitate integration
- Jointly worked to integrate the totality of the project
- Wrote this report

# CONCLUSION

This concludes the software part of the project for EEE4308 course. In this phase, we had to rely on sequential logic circuits heavily for a multitude of reasons, namely dice randomizer and Memory circuits.

We hope to proceed to the PCB segment of this project as soon as possible as that is the next step.